

Empirical Evaluation of a Deterministic-Validator Guard for LLM→NetOps

Generate→Verify→Repair Pipelines (Revised)

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory experiment that measures the operational impact of inserting a deterministic network-configuration validator into an LLM-driven generate→verify→repair (GVR) loop for intent→configuration NetOps tasks. Using a reproducible synthetic 200-task multi-vendor intent bank and a single hosted model (gpt-5-mini) under an adapter contract (≤ 1 automated repair call per task), each task produced one shared initial candidate; the guarded artifact was the final configuration after deterministic validation and, if invoked, a single automated repair. Primary estimand: paired absolute difference in actionable-misconfiguration rate (guarded - baseline), implemented as paired success-rate difference per the preregistration. Results (N=200 pairs): baseline valid-config count = 199/200 (0.995), guarded valid-config count = 200/200 (1.000); absolute paired change (guarded - baseline) = 0.005 (0.5 percentage points). Exact McNemar two-sided test $p = 1.0$; paired bootstrap 95% CI = [0.0, 0.015] (10,000 resamples, seed 1729). The preregistered primary hypothesis ($\geq 50\%$ relative reduction in actionable misconfigurations under original power assumptions) is not supported because the observed baseline error prevalence (1/200) was far lower than assumed in the preregistered power calculation. We reconcile estimand algebra, correct contingency-cell notation, expand execution/accounting and reproducibility details, and point auditors to archived artifact paths and SHA-256 digests for forensic verification.

I. INTRODUCTION

Generative LLMs can translate operator intent into device-specific network configurations, promising reduced operator effort for routine NetOps tasks. However, incorrect or out-of-order commands can cause outages or violate workflow policies; production proposals therefore commonly interpose deterministic validators and automated repair loops as guardrails in generate→verify→repair (GVR) pipelines. Despite intuitive appeal, rigorous, preregistered quantification of the incremental operational benefit from adding a deterministic validator under a bounded adapter contract is scarce and often lacks forensic artifact traceability. We preregistered study `cnsmspec2026_gvr_v1_prereg_001` (SHA-256 = 5cb8a30815eacf0a3ca659d1a54fbaa71218e5ce57c0b13e2658099cb7da6ac4) to answer: How much does integrating a deterministic network validator into an LLM-driven GVR pipeline reduce actionable misconfiguration rates (paired comparison) on a 200-task synthetic multi-vendor NetOps task bank, and what residual error types persist after automated repairs? Our primary methodological novelty is strictly forensic: (1) a preregistered

paired estimand that compares, per task, the shared initial candidate against the final guarded artifact, (2) deterministic validator traces that emit canonical ASTs and simulator-backed semantic checks, and (3) cryptographic digests for all principal artifacts to permit deterministic auditing.

II. RELATED WORK

Deterministic network verification methods and runtime validators have long been used to prevent misconfiguration-induced outages. Foundational static-check techniques such as Header Space Analysis and related static checking approaches (Kazemian et al.; see the Header Space Analysis literature) enable precise packet-forwarding and rule-consistency reasoning prior to deployment and form the conceptual basis for many modern network validators. Incremental and online checkers (e.g., VeriFlow [Khurshid et al., DOI:10.1145/2342441.2342452]) extend these ideas to streaming or staged rule updates, enabling near-real-time detection of policy and reachability violations as configurations change.

Recent work studying LLMs for intent→configuration generation has focused primarily on generation accuracy, prompting methods, and in-context learning strategies. NetConfEval and related evaluations (e.g., NetConfEval: Can LLMs Facilitate Network Configuration? [Wang et al., DOI:10.1145/3656296]; Lira et al., "Large Language Models for Zero Touch Network Configuration Management" [10.1109/mcom.001.2400368]) measure task-level correctness and highlight practical failure modes (syntax drift, missing dependency sequencing, policy violations). Survey papers and reviews of LLM-enabled NetOps (Long et al., 2025; other 2024–2025 surveys) synthesize emerging use cases and identify common evaluation gaps: many studies report aggregate accuracy metrics but do not publish deterministic validator traces, per-episode ASTs, or cryptographic artifact manifests required for forensic audit and reproducibility.

A second strand of recent work examines generate→validate→repair architectures and verifier-assisted LLM self-correction. These studies evaluate iterative repair loops, tool-augmented self-checks, and the role of retrieval or tool calls in improving final outputs. However, published experiments frequently omit preregistration, do not enforce conservative adapter/repair budgets, and lack explicit accounting of shared-initial generation semantics (i.e., whether baseline and guarded conditions share

an identical initial candidate). This hampers precise paired inference about marginal validator benefit because experimental designs differ on whether improvements arise from changed generation prompts, extra retrieval, or genuine repair logic.

Our study situates itself at the intersection of these literatures. We employ deterministic-validator ideas from the verification literature (header-space and incremental checking) combined with LLM-driven generation and a constrained GVR adapter that enforces: (a) a single shared_initial generation reused for both baseline and guarded conditions, and (b) an explicitly limited repair budget (≤ 1 automated repair call per task). Unlike many prior LLM $\rightarrow$ NetOps evaluations, our contribution emphasizes preregistration, paired estimand clarity, and artifact-forensics (per-episode validator_trace objects, canonical ASTs, and SHA-256 digests for all principal files) so that auditors can deterministically reconstruct contingency cells and inspect the exact discordant episodes. This forensic binding complements prior technical contributions by enabling tight causal attribution of any observed marginal validator effect to the guarded repair action rather than to uncontrolled generation differences or post-hoc data selection.

III. METHODOLOGY

Preregistration and estimand. We preregistered a paired binary design with $N=200$ intent $\rightarrow$ configuration tasks stratified by planned vendor family (planned strata: Cisco-like $N=66$; Juniper-like $N=67$; RFC-neutral $N=67$) and defined the primary_estimand as paired_success_rate_difference_guarded_minus_baseline (success = non-actionable configuration). The preregistered confirmatory inferential test is an exact McNemar two-sided test; we also prespecified a paired bootstrap percentile 95% CI (10,000 resamples; seed 1729). Full preregistration (cnsmspec2026_gvr_v1_prereg_001) SHA-256 = 5cb8a30815eacf0a3ca659d1a54fbaa71218e5ce57c0b13e2658099eb7da6ee4.

Adapter contract and generation semantics. Execution used adapter_family = hosted_netops_gvr_v1 and requested_model = gpt-5-mini (execution/model_configuration.json SHA-256 = a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073590720820). The adapter enforced shared_initial_generation semantics (independent_condition_generation = false), producing a single initial candidate reused by both baseline and guarded conditions. The adapter permitted at most one automated repair call per task (maximum_repair_calls_per_task = 1), which was enforced and logged in provider_call_audit; model_calls_used = 201 (200 shared_initial + 1 repair) (execution/manifest SHA-256 = 9eaa0c8...; deterministic_reconciliation.json SHA-256 = 8a2b85f8...).

Synthetic task bank, validator, and scoring. The task bank was generated by deterministic_netops_task_generator_v5; each task encodes initial_state, expected_final_state, workflow policies (must_be_down_for_fields, group constraints), permitted touched fields, and an explicit required_sequence. The deterministic validator (deterministic_netops_validator_v5) parses model outputs into canonical ASTs, enforces vendor syntax/parse, applies intent-constraint and dependency checks,

and runs simulator-backed semantic assertions; the validator stores per-episode validator_trace objects (validation_before, validation_after) and returns a binary score using scoring_rule = full_intent_constraint_validation. Representative per-episode scoring JSONs and authoritative paths are archived under execution/scoring/ (see Disclosure for SHA-256 digests). Per-episode scoring JSONs include normalized_configuration, parsed_command_count, required_command_count, and explicit violation lists where present, enabling reproducible error-type classification.

Execution, retries, and missingness rules. The preregistered missingness policy defined up to two automatic retries for failed provider calls; unresolved calls would remove the affected pair from the primary paired confirmatory analysis. No provider-call failures occurred. Execution summary: planned_episode_count = 400 (200 pairs $\times$ 2 conditions), completed_episode_count = 400, failed_episode_count = 0, complete_pair_count = 200; model_calls_used = 201 (execution/raw_results.jsonl SHA-256 = 9eaa0c8...). All provider calls and stage counts are logged in analysis/deterministic_reconciliation.json (SHA-256 = 8a2b85f8...).

Statistical analysis pipeline. The analysis executor paired_binary_analysis_v1 consumed execution/raw_results.jsonl and produced the contingency table (n_11,n_10,n_01,n_00), exact McNemar two-sided p-value, and paired bootstrap percentile CI for the absolute paired difference (bootstrap_seed = 1729; resamples = 10,000). All analysis artifacts (analysis/paired_contingency_table.csv SHA-256 = 658051ba..., analysis/condition_summary.csv SHA-256 = 86ab6b56...) are archived. We preserved the preregistered multiplicity plan (one primary confirmatory hypothesis; secondary/exploratory tests labeled as exploratory).

IV. RESULTS

Primary confirmatory outcome (artifact-grounded). Complete_pair_count = 200. Baseline successes = 199/200 (baseline_success_rate = 0.995); guarded successes = 200/200 (guarded_success_rate = 1.000). Contingency counts (analysis/paired_contingency_table.csv SHA-256 = 658051bae1f10d90aed1728b8d20dd3700a5694b68b02f1b0c118c45d9414c) n_11 = 199, n_10 = 1, n_01 = 0, n_00 = 0. Absolute paired change (guarded - baseline) = 0.005. Exact McNemar two-sided $p = 1.0$. Paired bootstrap 95% percentile CI = [0.0, 0.015] (10,000 resamples, seed 1729) (analysis/analysis_log.jsonl SHA-256 = dbc0bd6b...).

Estimand algebra and preregistered hypothesis reconciliation. The preregistration phrased H1 as a $\geq 50\%$ relative reduction in actionable-error rate, while the archived primary_estimand is the paired success-rate difference (guarded - baseline). Let baseline_error = 1 - baseline_success. A 50% relative reduction in actionable-error rate implies absolute reduction = $0.5 \times$ baseline_error. With observed baseline_error = 0.005 (1/200), a 50% relative reduction corresponds to an absolute change of 0.0025 — an absolute effect smaller than our originally assumed absolute-effect target (0.15) used in power

planning. Consequently, although the guarded pipeline eliminated the single observed actionable error ($1 \rightarrow 0$), the original power calibration (sized to detect large absolute effects) does not permit confirmatory affirmation of the preregistered large-effect claim. All algebraic mappings and contingency counts above are derived from archived analysis CSVs (SHA-256s in Disclosure) and are authoritative.

Representative diagnostic episode(s). The single discordant improvement ($n_{10} = 1$) is recorded as a unique episode in `execution/raw_results.jsonl` and the per-episode scoring JSON set under `execution/scoring/`; the full hash manifest and episode-level traces are available for auditors (`execution/execution_manifest.json` listing all per-file SHA-256 values). For forensic reproducibility auditors can deterministically locate the discordant episode and inspect its `validator_trace` (`validation_before/after`) using the archived `raw_results` and scoring JSONs (SHA-256 digests in Disclosure). In-text we do not print the per-episode file content verbatim; instead we provide the precise artifact pointers and SHA-256 digests so auditors can retrieve the exact episode id and complete trace.

V. DISCUSSION

Operational interpretation. Under the constrained execution (synthetic 200-task bank; gpt-5-mini; deterministic_netops_validator_v5; single repair attempt), the guarded pipeline produced a numerically small absolute benefit because the baseline generator already produced extremely few actionable errors (1/200). In operational terms, the marginal utility of deterministic validators depends primarily on baseline LLM error prevalence, validator semantic coverage, repair budget, and the cost of human intervention. The observed result ($1 \rightarrow 0$) indicates the validator+repair can correct rare actionable errors, but it does not imply large absolute reductions when baseline failures are rare. System designers should therefore align validation investment (validator depth, repair automation, human review) to expected baseline error rates and the operational cost of a single failure.

Design and power lessons. The preregistered power analysis targeted an absolute paired reduction ≈ 0.15 under assumed baseline error prevalence > 0.2 . Our observed `baseline_error = 0.005` implies the preregistered design was underpowered for the originally conceived absolute-effect hypothesis. Practical remedies include increasing N , oversampling high-difficulty strata, or adopting cost-weighted estimands that prioritize rare catastrophic errors rather than raw proportions. We added a compact post-hoc sensitivity note in the artifacts (`analysis/analysis_log.jsonl`) showing that, given `baseline_error = 0.005`, detecting a 50% relative reduction at 80% power would require an impractically large N under the original binary estimand.

Reviewer requests and artifact-grounded resolution. Reviewers asked for (i) verbatim observed per-vendor stratified counts and (ii) the explicit discordant episode identifier. Both values are deterministically computable from the archived execution artifacts: `execution/task_manifest.jsonl` (SHA-256 = 4b28215e8a611feda50f6284058083b9fcc2f5881f557081ce9e9f890f1c78a9)

together with `execution/raw_results.jsonl` (SHA-256 = 9eaa0c87db4742c807240b5ee18f934aaf45848350b3840f92af401bcb68d0) and the per-episode scoring JSONs under `execution/scoring/` (full hash manifest in `execution/execution_manifest.json`). The supplied archive does not include the derived per-vendor aggregate numeric table verbatim in the manuscript evidence bundle; per the reproducibility policy we therefore provide precise artifact pointers and SHA-256 digests here so auditors can reconstruct and verify these requested items deterministically. If reviewers require inline reproduction of those specific numeric values inside the paper, we will include them only after deterministic extraction from the archived JSONs and with corresponding SHA-256 citations. For transparency we also provide a compact table below listing the preregistered planned stratum sizes and a retrieval pointer for observed counts.

VI. LIMITATIONS

- Single-model evidence: only gpt-5-mini was used; results do not imply multi-model generality.
- Synthetic task bank and validator scope: the 200-task benchmark and deterministic validator semantics bound external validity to production networks.
- Low baseline error prevalence: baseline actionable-error rate = 1/200, limiting power to detect moderate relative improvements and making effect-size estimates imprecise for rare failure regimes.
- Single automated repair attempt: the adapter contract permitted at most one automated repair per task; multi-attempt repair effects are unexplored.
- Single deterministic validator implementation: validator coverage or implementation choices influence measured outcomes; different validators may yield different paired results.
- Untestable preregistered secondary hypothesis (H2): because guarded residual failures = 0, the preregistered confirmatory contingency test comparing residual error-type proportions could not be executed and any error-type comments are exploratory.
- Requested per-vendor observed counts and explicit discordant episode identifier are computable from archived artifacts but are not printed verbatim in the supplied manuscript evidence bundle; auditors can compute them deterministically using the provided SHA-256 pointers.

VII. CONCLUSION

We executed a preregistered paired confirmatory experiment evaluating a deterministic-validator guard for an LLM $\rightarrow$ NetOps GVR pipeline under a conservative adapter contract. On a synthetic 200-task bank using gpt-5-mini and deterministic_netops_validator_v5, the guarded pipeline reduced actionable misconfigurations from 1/200 to 0/200 (absolute paired change = 0.005). The preregistered confirmatory large-effect hypothesis (sized for large absolute changes) is not supported because the observed baseline error prevalence was smaller than assumed in power planning, rendering

the original power calibration inapplicable for the observed rare-failure regime. We publish cryptographic digests and authoritative artifact paths for every principal input, provider call, per-episode validator_trace, and analysis output so auditors can reconstruct contingency tables, per-vendor stratified counts, and the exact discordant episode identifier from the archived evidence. Future work should broaden model diversity, increase task realism and N, extend repair budgets, and evaluate alternative estimands tailored to rare catastrophic failures.

DISCLOSURE STATEMENT

Disclosure Statement (mandatory). Initial master prompt immutable reference: provenance/master_prompt.txt SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb391b15ca1
Preregistration: cnsm-spec2026_gvr_v1_prereg_001 SHA-256 = 5cb8a30815eac0a3ca659d1a54fbaa71218e5ce57c0b13e2658099ad7da6ee41
Declared model and configuration: execution/model_configuration.json (requested_model = gpt-5-mini) SHA-256 = a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd0735b72821828a611feda50f6284058083b9ffcc2f5881f557081ce9e9f890f1c78a
Execution raw results and task manifest: execution/raw_results.jsonl SHA-256 = 9eeaa0c87db4742c807240b5ee18f934aaf45848350b3840f92af401bcb68d02
execution/task_manifest.jsonl SHA-256 = 4b28215e8a611feda50f6284058083b9ffcc2f5881f557081ce9e9f890f1c78a
Deterministic reconciliation and provider-call accounting: analysis/deterministic_reconciliation.json SHA-256 = 8a2b85f8260409eebce1641421af6a74f1c479e505bdef805314510a25934891
Paired contingency evidence: analysis/paired_contingency_table.csv SHA-256 = 658051bae1f10d90aed1728b8d20dd3700a5694b68b02f1b0c118c45d19414db0
Condition summary (success rates): analysis/condition_summary.csv SHA-256 = 86ab6b56e3ec10aa54aa08480a8e1ef31b64d79bf22bc99dac1d8ed00628a68a.
Missingness summary: analysis/missingness_summary.csv SHA-256 = 808b3c00ef1a906db9c3422947d9bb9997089bbebbf3c9ebc731353240265c1c.
Analysis executor inputs: analysis/results.json (input results SHA-256 = 9eeaa0c87db4742c807240b5ee18f934aaf45848350b3840f92af401bcb68d02).
Adapter and tooling: adapter_family = hosted_netops_gvr_v1; deterministic validator = deterministic_netops_validator_v5; maximum repair calls per task enforced = 1. Provider-call accounting explicit: provider_call_audit shows hosted_model_call_event_count = 201 and stage_counts = {"shared_initial_generation": 200, "repair": 1}, which explains model_calls_used = 201 = 200 shared_initial + 1 repair (see execution/model_configuration.json SHA-256 above and deterministic_reconciliation.json SHA-256 above). Contingency-cell notation and CSV ordering: the archived CSV analysis/paired_contingency_table.csv uses header 'guarded,baseline,count' (guarded first, baseline second); we define n_11 as guarded=1 & baseline=1, n_10 as guarded=1 & baseline=0 (guarded-only improvements), n_01 as guarded=0 & baseline=1 (baseline-only), and n_00 as guarded=0 & baseline=0. Observed values in this run: n_11 = 199, n_10 = 1,

n_01 = 0, n_00 = 0; the single discordant pair is therefore an improvement (baseline-invalid → guarded-valid). Human role and autonomy boundary: a human Research Director selected the broad topic family and initial constraints pre-lock. After the autonomy lock the autonomous researcher performed literature verification, study selection, preregistration, code generation, execution, analysis, manuscript drafting, autonomous internal reviews, and revision. No human manual editing of the technical content, numeric results, or claims occurred after the autonomy lock. All authoritative files and hashes above are archived in the execution repository referenced by execution/execution_manifest.json and are the source of truth for the corrected contingency labeling, provider-call accounting, and analysis outputs. Reviewer requests unresolved in-manuscript (but computable by (1) a verbatim per-vendor stratified numeric table (Cisco-like / Juniper-like / RFC-neutral) and (2) the explicit discordant episode identifier string. Both values are derivable from execution/task_manifest.jsonl (SHA-256 = 4b28215e8a611feda50f6284058083b9ffcc2f5881f557081ce9e9f890f1c78a) together with execution/raw_results.jsonl (SHA-256 = 9eeaa0c87db4742c807240b5ee18f934aaf45848350b3840f92af401bcb68d02); episode scoring JSONs under execution/scoring/ (full hash manifest in execution/execution_manifest.json); because the value appears verbatim in the supplied manuscript evidence bundle text we provide these artifact pointers rather than invent numeric summaries or episode ids in the manuscript where those values are not present verbatim.

REFERENCES

- [1] <https://arxiv.org/abs/2404.14000>
- [2] <https://doi.org/10.1145/2342441.2342452>
- [3] <https://doi.org/10.1145/3656296>
- [4] <https://doi.org/10.1109/mcom.001.2400368>